Esercitazione di laboratorio IoT – Livello molto avanzato

Titolo: Progettazione e implementazione di un sistema di sicurezza elettronico con autenticazione, controllo degli accessi e rilevamento delle intrusioni

1. Obiettivo dell'esercitazione

Si vuole sviluppare un sistema di sicurezza elettronico intelligente, in grado di controllare l'accesso a un ambiente protetto attraverso un codice segreto, rilevare tentativi di intrusione e gestire automaticamente differenti livelli di allarme.

Il sistema dovrà utilizzare un ESP32 programmato in MicroPython e dovrà integrare meccanismi di autenticazione, controllo degli attuatori, gestione degli eventi e temporizzazioni.

L'esercitazione deve essere svolta esclusivamente collegando i componenti all'ESP32 mediante breadboard e cavetti, senza realizzare strutture meccaniche, modellini o prototipi fisici.

Il servomotore e il motore stepper simuleranno due differenti meccanismi di blocco di un sistema di accesso. Non è richiesta alcuna connessione Wi-Fi.

2. Componenti disponibili

- ESP32
- LED
- Fotoresistore
- Potenziometro
- Servomotore SG90
- Motore stepper 28BYJ-48 con driver ULN2003
- Pulsante
- Buzzer passivo

3. Specifiche funzionali

Parte A – Inserimento e verifica del codice segreto

Il sistema deve prevedere un meccanismo di autenticazione basato su un codice segreto composto da quattro cifre, ciascuna compresa tra 0 e 9.

Il codice iniziale è:

3 – 7 – 1 – 9

L'utente deve inserire il codice utilizzando esclusivamente il potenziometro e il pulsante.

Il valore acquisito dal potenziometro, normalizzato nell'intervallo 0–4095, deve essere convertito in una cifra decimale secondo la seguente formula:

$$cifra = \min\left(9, \left\lfloor \frac{ADC \cdot 10}{4096} \right\rfloor\right)$$

Il LED deve fornire un'indicazione visiva della cifra selezionata attraverso una sequenza di lampeggi.

Cifra selezionata	Segnalazione LED
0	Un lampeggio lungo di 800 ms
1	Un lampeggio breve
2	Due lampeggi brevi
...	...
9	Nove lampeggi brevi

Ogni lampeggio breve deve durare 100 ms, con una pausa di 150 ms tra lampeggi consecutivi.

Una pressione breve del pulsante conferma la cifra selezionata.

Dopo l'inserimento della quarta cifra, il sistema deve verificare automaticamente la correttezza del codice.

Vincoli aggiuntivi:

1. Ogni cifra deve essere memorizzata in una struttura dati.
2. Il sistema deve gestire correttamente anche la cifra 0.
3. Se non viene confermata alcuna cifra per 15 secondi, l'inserimento deve essere annullato.
4. Se il potenziometro cambia posizione durante la sequenza di lampeggi, la sequenza corrente deve terminare e deve essere avviata quella corrispondente alla nuova cifra.
5. Il programma deve evitare che piccole oscillazioni dell'ADC producano cambiamenti continui della cifra selezionata, introducendo un'isteresi di 30 unità ADC sui confini tra cifre.

Parte B – Gestione dell'autenticazione

Il sistema deve prevedere tre possibili risultati dell'autenticazione.

Condizione	Comportamento
Codice corretto	Accesso autorizzato
Primo codice errato	Segnalazione di errore
Tre codici errati consecutivi	Blocco temporaneo del sistema

Quando il codice è corretto:

- Il buzzer emette due beep a 1500 Hz, ciascuno della durata di 150 ms.
- Il LED rimane acceso per 2 secondi.
- Il sistema avvia la procedura di sblocco descritta nella Parte C.
- Il contatore dei tentativi falliti viene azzerato.

Quando il codice è errato:

- Il buzzer emette un beep a 400 Hz per 500 ms.
- Il LED esegue tre lampeggi rapidi.
- Il sistema incrementa il contatore dei tentativi falliti.
- Il codice inserito viene cancellato.

Al terzo errore consecutivo il sistema entra nello stato BLOCCATO per 30 secondi.

Durante questo intervallo:

- Non è possibile inserire nuovi codici.
- Il LED lampeggia con una frequenza di 2 Hz.
- Il buzzer emette un beep a 800 Hz ogni 5 secondi.
- I due motori devono rimanere nella configurazione di blocco.

Al termine dei 30 secondi il sistema torna automaticamente a consentire l'autenticazione e azzera il contatore dei tentativi falliti.

Parte C – Sistema di blocco elettronico a due attuatori

L'accesso all'ambiente protetto è controllato da due attuatori indipendenti:

- Il servomotore rappresenta il blocco principale.
- Il motore stepper rappresenta il blocco secondario.

Il sistema deve rispettare una sequenza obbligatoria di sblocco.

Procedura di apertura

1. Il buzzer segnala l'avvenuta autenticazione.
2. Il servomotore si sposta gradualmente da 0° a 90°.
3. Soltanto quando il servo raggiunge 90°, lo stepper esegue 1024 half-step in senso orario.
4. Al termine, il sistema entra nello stato ACCESSO_CONSENTITO.
5. Il LED rimane acceso per indicare che il sistema è sbloccato.

Il servo deve muoversi con incrementi massimi di 3° ogni 30 ms.

Lo stepper deve essere comandato utilizzando la sequenza half-step e un intervallo minimo di 3 ms tra due passi consecutivi.

Procedura di chiusura

Dopo 10 secondi dall'avvenuto sblocco, il sistema deve avviare automaticamente la chiusura.

La procedura deve essere eseguita nell'ordine inverso:

1. Lo stepper esegue 1024 half-step in senso antiorario.
2. Soltanto dopo il completamento dello stepper, il servo ritorna gradualmente a 0°.
3. Il LED si spegne.

4. Il sistema torna nello stato di attesa del codice.

Il sistema non deve avviare un nuovo processo di autenticazione durante le procedure di apertura e chiusura.

Vincolo fondamentale: nessuna operazione di movimentazione deve bloccare la lettura dei sensori o il controllo degli allarmi.

Parte D – Rilevamento delle intrusioni

Il fotoresistore deve essere utilizzato come sensore per il rilevamento di aperture non autorizzate dell'ambiente protetto.

Si ipotizzi che, in condizioni normali, il fotoresistore sia mantenuto al buio e che un aumento improvviso della luminosità simuli un tentativo di intrusione.

All'avvio del sistema, il programma deve acquisire 20 campioni del fotoresistore e calcolarne la media, ottenendo un valore iniziale di riferimento \(L_0\).

Il valore del fotoresistore deve essere successivamente acquisito ogni 50 ms.

Si definisce:

$$\Delta L = L_{\text{attuale}} - L_0$$

Il sistema deve distinguere tre condizioni.

Variazione di luminosità	**Condizione**
DeltaL < 500	Normale
500 <= Delta L < 1200	Sospetta
DeltaL >= 1200	Possibile intrusione

Il comportamento richiesto è il seguente:

- Se la condizione sospetta permane per almeno 3 secondi consecutivi, il sistema entra in PREALLARME.
- Se la condizione di possibile intrusione permane per almeno 1 secondo consecutivo, il sistema entra in ALLARME.
- Se la luminosità ritorna normale prima della scadenza dell'intervallo previsto, il relativo contatore temporale deve essere azzerato.

Il rilevamento delle intrusioni deve essere attivo soltanto quando il sistema è protetto, ossia durante l'attesa del codice e il blocco temporaneo per tentativi falliti.

Durante l'accesso autorizzato deve essere disabilitato, per evitare falsi allarmi.

Parte E – Gestione del preallarme e dell'allarme

PREALLARME

Quando viene rilevata una condizione sospetta:

- Il LED deve lampeggiare con un periodo di 1000 ms.
- Il buzzer deve emettere un beep a 1000 Hz ogni 2 secondi.
- Il sistema deve continuare a monitorare il fotoresistore.

Se la luminosità ritorna normale per almeno 2 secondi consecutivi, il preallarme deve terminare.

Se, durante il preallarme, viene rilevata una possibile intrusione per almeno 1 secondo, il sistema deve passare immediatamente ad ALLARME.

ALLARME

Quando viene rilevata un'intrusione:

- Il LED deve lampeggiare rapidamente, con periodi di 200 ms.
- Il buzzer deve alternare segnali a 1000 Hz e 2000 Hz ogni 250 ms.
- Il sistema deve impedire qualsiasi procedura ordinaria di apertura.
- I motori devono mantenere o raggiungere, secondo una sequenza non bloccante, la configurazione di blocco.

Il sistema deve memorizzare la causa dell'allarme.

L'allarme non deve cessare automaticamente quando la luminosità ritorna normale.

Per disattivarlo è necessario inserire correttamente il codice segreto.

Durante ALLARME, l'inserimento del codice rimane disponibile: LED e buzzer continuano a segnalare l'allarme, mentre la cifra selezionata viene visualizzata sul terminale seriale. La conferma avviene tramite pulsante.

La corretta autenticazione disattiva l'allarme e riporta il sistema in attesa del codice, senza avviare l'apertura. Se la condizione di intrusione persiste, l'allarme potrà riattivarsi secondo le normali temporizzazioni.

Parte F – Modifica del codice segreto

Il sistema deve consentire la modifica del codice segreto esclusivamente dopo un'autenticazione valida.

Durante lo stato ACCESSO_CONSENTITO, una pressione lunga del pulsante di almeno 3 secondi deve attivare la modalità CAMBIO_CODICE.

In questa modalità, l'utente deve:

1. Inserire il nuovo codice di quattro cifre.
2. Confermare ogni cifra mediante il pulsante.
3. Reinserire il codice una seconda volta.
4. Attendere la verifica di corrispondenza.

Se i due codici coincidono, il nuovo codice sostituisce quello precedente.

Il buzzer deve produrre tre beep a 1800 Hz da 100 ms ciascuno.

Se i codici non coincidono, la modifica deve essere annullata e il sistema deve mantenere il codice precedente.

Il cambio del codice deve completarsi entro 30 secondi. In caso contrario, la procedura viene annullata.

Durante CAMBIO_CODICE, la chiusura automatica viene sospesa; al termine o all'annullamento della procedura, deve riprendere con un nuovo intervallo di 10 secondi.

La modifica deve essere conservata in memoria durante l'esecuzione del programma. Non è richiesta la persistenza dopo lo spegnimento dell'ESP32.

Parte G – Gestione degli eventi simultanei

Il sistema deve poter gestire eventi che si verificano contemporaneamente.

Si considerino, ad esempio, le seguenti situazioni:

1. Il fotoresistore rileva un'intrusione durante l'inserimento di un codice.
2. Il pulsante viene premuto mentre il buzzer sta producendo un segnale.
3. La luminosità cambia durante la movimentazione dello stepper.
4. Il tempo di blocco per tentativi falliti termina mentre è presente una condizione di intrusione.
5. Il timeout di inserimento del codice scade durante una sequenza di lampeggi.

Lo studente deve definire e implementare una politica di gestione delle priorità.

La priorità più alta deve essere assegnata agli eventi di sicurezza, seguiti dal controllo degli attuatori e infine dalle segnalazioni ordinarie.

Le attività con priorità inferiore devono essere interrompibili o rinviabili senza compromettere la correttezza del funzionamento.

4. Vincoli di implementazione

Il programma deve essere strutturato in funzioni, ad esempio:

```
def leggi_sensori():
    pass


def acquisisci_cifra():
    pass


def verifica_codice():
    pass


def gestisci_autenticazione():
    pass
```

```
def gestisci_servo():
    pass

def gestisci_stepper():
    pass

def rileva_intrusione():
    pass

def gestisci_allarmi():
    pass

def gestisci_buzzer():
    pass

def gestisci_pulsante():
    pass

def aggiorna_stato():
    pass
```

Gli studenti devono rispettare le seguenti condizioni:

1. Non utilizzare cicli for o while bloccanti per completare le rotazioni dei motori.
2. Utilizzare time.ticks_ms() e time.ticks_diff() per gestire tutte le temporizzazioni.
3. Implementare il debouncing del pulsante.
4. Gestire gli stati del sistema mediante una macchina a stati.
5. Memorizzare la posizione corrente del servo e dello stepper.
6. Evitare ripetizioni indesiderate delle operazioni.
7. Garantire la lettura periodica dei sensori durante ogni fase.
8. Gestire le sequenze luminose e acustiche senza interrompere il programma principale.
9. Prevedere un meccanismo di timeout per l'inserimento del codice.
10. Garantire che nessuna procedura ordinaria possa compromettere la gestione degli allarmi.

5. Prove di funzionamento richieste

Test	Operazione	Risultato atteso
1	Inserire il codice corretto	Apertura sequenziale dei due blocchi
2	Inserire un codice errato	Segnalazione acustica e incremento contatore
3	Inserire tre codici errati	Blocco per 30 secondi
4	Tentare l'inserimento durante il blocco	Nessuna autenticazione consentita
5	Attendere 10 secondi dopo l'apertura	Avvio della chiusura automatica
6	Aumentare moderatamente la luminosità per 3 secondi	Attivazione PREALLARME
7	Aumentare notevolmente la luminosità per 1 secondo	Attivazione ALLARME
8	Ripristinare la luminosità dopo un allarme	L'allarme rimane attivo
9	Inserire il codice corretto durante ALLARME	Disattivazione dell'allarme
10	Modificare il codice dopo autenticazione	Memorizzazione del nuovo codice
11	Reinserire un codice diverso durante la conferma	Modifica annullata
12	Interrompere l'inserimento del codice per 15 secondi	Annullamento automatico
13	Modificare la luminosità durante il movimento dei motori	Acquisizione periodica garantita
14	Premere il pulsante durante una segnalazione acustica	Evento riconosciuto correttamente
15	Eseguire ripetutamente apertura e chiusura	Nessuna perdita della posizione logica dei motori

6. Consegna finale

Lo studente deve consegnare:

- Il codice MicroPython completo e commentato.
- Una fotografia del circuito funzionante su breadboard.
- Un diagramma della macchina a stati utilizzata.
- Una breve descrizione del meccanismo di autenticazione, delle temporizzazioni e della gestione degli eventi simultanei.
- I risultati delle prove effettuate.

La principale difficoltà dell'esercitazione consiste nel progettare un sistema embedded in cui autenticazione, controllo degli accessi, rilevamento delle intrusioni e gestione degli attuatori operino contemporaneamente, garantendo il rispetto delle priorità di sicurezza e senza utilizzare operazioni bloccanti.

Questa traccia richiede quindi non soltanto la conoscenza dei singoli componenti hardware, ma anche la capacità di progettare un'applicazione embedded articolata, affidabile e correttamente sincronizzata.